

10주차 예습과제

📖 자연어처리와 컴퓨터비전 심층학습 5~6장(~TF-IDF)

5. 토큰화

- 자연어(Natural Language)

- 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어 (↔인공적으로 만들어진 프로그래밍 언어)

- 자연어 처리(Natural Language Processing, NLP)

- 컴퓨터가 인간의 언어 이해, 해석 및 생성하기 위한 기술
- 컴퓨터가 인간과 유사한 방식으로 인간의 언어 이해, 처리하는 것이 주요 목표
- 인간 언어의 구조, 의미, 맥락 분석하고 이해할 수 있는 알고리즘과 모델 개발

- 자연어 처리 모델이 해결해야 할 문제

- 모호성: 다양한 의미 이해하고 명확하게 구분
- 가변성: 다양한 강세, 사투리, 신조어 등의 가변성을 처리
- 구조: 문장의 구조와 문법적 요소 이해하여 의미 추론, 분석

→ 말뭉치(Corpus)를 일정한 단위인 **토큰(Token)**으로 나누어야 함

- 말뭉치: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어

- 토큰: 개별 단어나 문장 부호와 같은 텍스트, 말뭉치보다 더 작은 단위

- **토큰화(Tokenization)**: 컴퓨터가 자연어를 이해할 수 있게 토큰으로 나누는 과정

- 토큰화를 위해 **토큰나이저(Tokenizer)** 사용

- 토큰나이저: 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어

▼ 토큰나이저를 사용한 토큰화 예시

입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'

결과: ['형태소', '분석기', '를', '이용', '하', '여', '간단', '하', '게', '토큰', '화', '하', '르', '수', '있', '다', '.']

- 토큰라이저를 구축하는 방법
 - 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
 - 정규표현식 적용: 정규 표현식으로 특정 패턴 식별해 텍스트 분할
 - 어휘 사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용
 - 머신러닝 활용: 데이터세트를 기반으로 토큰화하는 방법 학습한 머신러닝 적용
 - 어휘 사전(**Vocabulary, Vocab**)은 사전에 정의된 단어 활용해 토큰라이저 구축하는 방법
 - 없는 단어나 토큰, 즉 **OOV(Out of Vocab)** 존재할 수 있음
 - 큰 어휘 사전 구축 시 학습 비용의 증대, **차원의 저주(Curse of Dimensionality)** 발생 가능
 - 모든 토큰이나 단어를 벡터화 시 어휘 사전 속 토큰 개수만큼의 차원 필요
 - 벡터값이 거의 모두 0의 값 가지는 **희소(sparse)** 데이터로 표현됨
-

단어 및 글자 토큰화

- 토큰화
 - 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업
 - 정확한 분석에 필요, 단어나 문장의 빈도수와 출현 패턴 등 파악 가능

단어 토큰화

- 단어 토큰화(**Word Tokenization**)
 - 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
 - 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자 활용
 - 가장 일반적인 토큰화 방법, 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용



['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']

- ## 글자 토큰화

- ## #글자 토큰화

['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', ' ',

- 3



◦ 자모(jamo) 라이브러리 활용

- 한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

```
#자모 변환 함수
retval = jamo.h2j(
    hangul_string
)
```

- 입력된 한글 문자열을 유니코드 U+1000~U+11FE 사이 조합형 한글 자모로 변환
 - 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 조합
 - 완성형: 조합된 글자 자체에 값 부여해 인코딩
- `h2j` 함수는 완성형으로 입력된 한글을 조합형 한글로 변환

```
#한글 호환형 자모 변환 함수
retval = jamo.j2hcj(
    jamo
)
```

- `j2hcj` 함수는 조합형 한글 문자열을 자소 단위로 나눠 반환
- 조합형 한글로 입력된 문자열 → 초성, 중성, 종성으로 나뉨
- 완성형 한글을 쉽게 자소 단위로 나눌 수 있음



```
from jamo import h2j, j2hcj
```

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

```
['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅊ', ' ', 'ㄱ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄴ', ' ', 'ㅂ',
```

- 자모 변환 함수, 한글 호환성 자모 변환 함수 활용해 자소 단위로 분해

- 글자 단위 토큰화 장점

- 단어 단위 토큰화에 비해 적은 크기의 단어 사전 구축 가능
- '도', '는', '.' 과 같은 접사와 문장부호의 의미 학습 가능
- 적은 크기의 단어 사전으로도 OOV 줄일 수 있음
- 글자 단위 토큰화 단점
 - 개별 토큰은 의미가 없으므로 자연어 모델이 각 토큰의 의미 조합해 결과 도출해야 함
 - 토큰 조합 방식 사용해 문장 생성, **개체명 인식(Name Entity Recognition)** 등 구현할 경우, 다의어나 동음이의어 많은 도메인에서는 구별 어려움
 - 모델 입력 **시퀀스(sequence)**의 길이 길어질수록 연산량 증가
Ex) 리뷰 예시에서 입력 시퀀스 길이 (단어토큰화 13, 글자 토큰화 53, 자소 단위 토큰화 101)

형태소 토큰화

- **형태소 토큰화(Morpheme Tokenization)**
 - 텍스트를 형태소 단위로 나누는 토큰화 방법
 - 한국어와 같은 교착어(Agglutinative Language)인 언어에서 중요하게 수행

형태소 어휘 사전

- **형태소 어휘 사전(Morpheme Vocabulary)**
 - 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전

Ex) 형태소 단위로 어휘 사전 구축 시 ['-는', '-에게', '그', '나']와 같은 정보에서 '그녀'의 토큰 정보만 새로 학습 해 '그녀는'이나 '그녀에게'와 같은 어휘를 쉽게 학습할 수 있음

- 각 형태소가 어떤 품사에 속하는지와 해당 품사의 뜻 등의 정보도 함께 제공
- **품사 태깅(POS Tagging)**
 - 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 **품사(Part Of Speech, POS)**를 태깅하는 작업
 - 문맥을 고려할 수 있어 더욱 정확한 분석이 가능

Ex) “그는 나에게 인사를 했다”

→ 그(명사)+는(조사)+나(명사)+에게(조사)+인사(명사)+를(조사)+했다(동사)

- KoNLPy, NLTK, spaCY 라이브러리 등

KoNLPy

- 한국어 자연어 처리 위해 개발된 라이브러리
 - 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
 - 텍스트 데이터 전처리, 분석을 위한 다양한 도구와 함수를 제공해 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용
- 자바 언어 기반 한국어 형태소 분석기로, **자바 개발 키트(Java Development Kit, JDK)** 기반으로 개발
- **Okt(Open Korean Text), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab)** 등의 다양한 형태소 분석기 지원
 - 예제에서는 SNS 텍스트 데이터 기반으로 개발된 Okt와 국립국어원에서 배포한 세종 말뭉치를 기반으로 학습된 꼬꼬마 사용



```
#Okt 토큰화
from konlpy.tag import Okt
```

```
okt = Okt()
```

```
sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."
```

```
nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)
```

```
print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출 : ['무엇', '이든', '상상', '할', '수', '있는', '사람', '은', '무엇', '이든', '만들어', '낼', '수', '있다', '.']
품사 태깅 : [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'), ('있는', 'Adjective'), ('사람', 'Noun'),
```

- **Okt** 객체
 - 문장을 입력받아 명사, 구, 형태소, 품사 등의 정보를 추출하는 여러 메서드 제공
 - 대표적인 메서드는 **명사 추출(`okt.nouns`)**, **구문 추출(`okt.phrases`)**, **형태소 추출(`okt.morphs`)**, **품사 태깅(`okt.pos`)**
 - 입력된 문장에서 각각 명사, 어절 구 단위, 형태소만 추출해 리스트 반환
 - 품사 태깅은 입력된 문장에서 각 단어에 대한 품사 정보를 추출하여 (형태소, 품사) 형태의 튜플로 구성된 리스트 반환



#꼬꼬마 토큰화

```
from konlpy.tag import Kkma
```

```
kkma = Kkma()
```

```
sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."
```

```
nouns = kkma.nouns(sentence)
```

```
sentences = kkma.sentences(sentence)
```

```
morphs = kkma.morphs(sentence)
```

```
pos = kkma.pos(sentence)
```

```
print("명사 추출 :", nouns)
```

```
print("구 추출 :", phrases)
```

```
print("형태소 추출 :", morphs)
```

```
print("품사 태깅 :", pos)
```

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇']
```

```
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
```

```
형태소 추출 : ['무엇', '이', '든', '상상', '하', 'ㄴ', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어', '내', 'ㄴ', '수', '있', '다', '.']
```

```
품사 태깅 : [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄴ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'),
```

◦ Kkma 클래스

- 명사, 문장, 형태소, 품사 추출 가능
- 구문 추출 기능은 지원 X, **명사 추출(`kkma.sentences`)** 기능을 제공

- 동일한 메서드라도 형태소 분석기의 특징에 따라 다른 결과

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

19개 품사 태깅

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

56개 품사 태깅

NLTK

- **NLTK(Natural Language Toolkit)**
 - 자연어 처리 위해 개발된 라이브러리
 - 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공
- 예제에서는 Punkt 모델, **Averaged Perceptron Tagger** 모델 활용해 토큰화 및 품사 태깅 작업을 수행
 - 두 모델 모두 **트리뱅크(Treebank)**라는 대규모의 영어 말뭉치 기반 학습

- Punkt 모델은 통계 기반 모델, Averaged Perceptron Tagger는 퍼셉트론 기반 품사 태깅 수행



```
#패키지 및 모델 다운로드
import nltk

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
```

- punkt 모델과 averaged_perceptron_tagger 모델 다운로드



```
#영문 토큰화
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible"

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)
```

```
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible']
['Those who can imagine anything, can create the impossible']
```

- 단어 토크나이저(word_tokenize)
 - 문장을 입력받아 공백 기준으로 단어를 분리
 - 구두점 등을 처리해 각각의 단어(token)를 추출해 리스트로 반환
- 문장 토크나이저(sent_tokenize)
 - 문장을 입력받아 마침표, 느낌표, 물음표 등의 구두점 기준으로 문장을 분리해 리스트로 반환



```
#영문 품사 태깅
from nltk import tag
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible"

word_tokens = tokenize.word_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(pos)

[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), ('', ','), ('', ','), ('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ')]
```

◦ 품사 태깅(**pos_tag**)

- 토큰화된 문장에서 품사 태깅을 수행
- 토큰화된 단어들이 들어가야 하며, Averaged Perceptron Tagger 모델이 설치되어 있어야 함

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

35개 품사 태깅

spaCy

- **spaCy**
 - 사이썬(Cython) 기반으로 개발된 오픈 소스 라이브러리로, 자연어 처리 위한 기능 제공
 - 빠른 속도, 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리
 - NLTK에서 사용하는 모델보다 더 크고 복잡, 더 많은 리소스 요구
- spaCy는 GPU 가속을 비롯해 24개 이상의 언어로 사전 학습된 모델을 제공
- 예제에서는 영어로 사전 학습된 모델인 `en_core_web_sm` 을 설치



```
#spaCy 품사 태깅
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible"
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

```
[PRON - DT ] : Those
[PRON - WP ] : who
[AUX - MD ] : can
[VERB - VB ] : imagine
[PRON - NN ] : anything
[PUNCT - , ] : ,
[AUX - MD ] : can
[VERB - VB ] : create
[DET - DT ] : the
[ADJ - JJ ] : impossible
```

- 사전 학습된 모델을 기반으로 처리, **spaCy 모델 불러오기 함수(`load`)**를 통해 모델 설정
- **객체 지향적(Object Oriented)**으로 구현돼 처리한 결과를 `doc` 객체에 저장
 - `doc` 객체는 다시 여러 `token` 객체로 이뤄져 있으며, 이 `token` 객체에 대한 정보 기반으로 다양한 자연어 처리 작업 수행
 - `nlp` 인스턴스에 문장 입력하면 `doc` 객체가 반환되며 `token` 객체의 `tag_` 나 `pos_` 와 같은 속성에 접근해 값 확인 가능
- `token` 객체
 - 기본 품사 속성(`pos_`), 세분화 품사 속성(`tag_`), 원본 텍스트 데이터(`text`), 토큰 사이의 공백을 포함하는 텍스트 데이터(`text_with_ws`), 벡터(`vector`), 벡터 노름(`vector_norm`) 등의 속성 포함돼 있음

표 5.4 spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

- 형태소 분석과 품사 태깅은 문장에서 의미 갖는 최소 단위이므로 자연어 처리에서 매우 중요한 전처리 작업이다

하위 단어 토큰화

- 형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에 취약
 - 잠재적으로 어휘 사전의 크기 크게 만들고, OOV에 대응하기 어렵게 함
 - 현대 자연어 처리에서는 신조어의 발생, 오타자, 축약어 등으로 인해 분석할 단어의 양 많아지는 문제 발생

- 하위 단어 토큰화(Subword Tokenization)

- 하나의 단어가 빈번하게 사용되는 **하위 단어(Subword)**의 조합으로 나누어 토큰화하는 방법
- 단어의 길이 줄일 수 있어 처리 속도 빨라짐, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제의 완화
- 바이트 페어 인코딩, 워드피스, 유니그램 모델 등

바이트 페어 인코딩

- 바이트 페어 인코딩(Byte Pair Encoding, BPE)

- 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합 찾아 부호화하는 압축 알고리즘
- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합탐지와 부호화 반복
- 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현



원문: abracadabra

Step 1: AracadAra (ab→A)

Step 2: ABcadAB (ra→B)

Step 3: Ccadc (AB→C)



빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)

어휘 사전: ['l', 'o', 'w', 'e', 'r', 'n', 'e', 'w', 'e', 's', 't', 'w', 'i', 'd', 'e', 's', 't']

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)

어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)

어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'e', 's']

(→ 'e', 's' 쌍이 총 9번으로 가장 많이 등장, 빈도 사전에서 'es'로 병합하고 어휘 사전에 'es' 추가)

(동일한 과정을 반복)

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)

어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'e', 's', 'e', 's', 't', 'l', 'o', 'l', 'o', 'w', 'n', 'e', 'w', 'e', 's', 't', 'w', 'i', 'w', 'i', 'd', 'e', 's', 't']

센텐스피스

- 예제에서는 **센텐스피스(Sentencepiece)**와 **코포라(Korpora)** 라이브러리를 활용해 토큰라이저를 학습
- 센텐스피스 라이브러리
 - 구글에서 개발한 오픈소스 하위 단어 토큰라이저 라이브러리
 - 바이트 페어 인코딩과 유사한 알고리즘 사용해 입력 데이터 토큰화, 단어 사전 생성
 - 워드피스, 유니코드 기반의 다양한 알고리즘과 사용자가 직접 설정할 수 있는 하이퍼파라미터로 세밀한 토큰라이징 기능을 제공
- 코포라 라이브러리
 - 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리

토큰라이저 모델 학습

- 센텐스피스 라이브러리를 통해 바이트 페어 인코딩을 수행하는 토큰라이저 모델 학습



```
#청와대 청원 데이터 다운로드
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일 :", petition.begin)
print("청원 종료일 :", petition.end)
print("청원 동의 수 :", petition.num_agree)
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

청원 시작일 : 2017-08-25
 청원 종료일 : 2017-09-24
 청원 동의 수 : 88
 청원 범주 : 육아/교육
 청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.
 청원 본문 : 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비

- 말뭉치 불러오기(`Korpora.load`)를 통해 청와대 청원 말뭉치 데이터 다운로드
 - `train` 속성으로 학습 데이터 가지고 올 수 있음
 - 첫 번째 청원 데이터는 `dataset[0]` 으로 불러옴
 - `petition` 에 청원 시작일, 동의 수, 제목 등의 정보가 포함



```
#학습 데이터셋 생성
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open("../datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

- `get_all_texts` 메서드로 본문 데이터셋 한 번에 불러오고 하나의 텍스트 파일로 저장



```
#토큰나이저 모델 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 model_type=bpe"
)
```

- **SentencePieceTrainer**
 - 센텐스피스 라이브러리는 **SentencePieceTrainer** 클래스의 **Train** 메서드로 토큰나이저 모델을 학습
 - 센텐스피스는 문자열 입력을 통해 인자들을 전달받음

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰나이저가 다룰 수 있는 글자의 비율

model_type	토큰나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

- 토큰나이저 모델 학습 완료되면 **petition_bpe.model** 파일과 **petition_bpe.vocab** 파일이 생성
 - **model** 파일은 학습된 토큰나이저, **vocab** 파일은 어휘 사전이 저장됨
- 어휘 사전 파일에 밑줄 문자가 포함된 데이터를 볼 수 있음
 - 센텐스피스 라이브러리는 띄어쓰기나 공백도 특수문자로 취급해 토큰화 과정에서 '_'로 공백을 표현함



```
#바이트 페어 인코딩 토큰화
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있습니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

단일 문장 토큰화 : ['안녕하세요', ' ', ' ', ' ', '토', '크', '나', '이', '저', '가', ' ', '잘', ' ', '학', ' ', '습', ' ', '되', '었', ' ', '군', '요', '!', '!']
여러 문장 토큰화 : [['이렇게', ' ', '입', ' ', '력', ' ', '값', '을', ' ', '리', ' ', '스', '트', ' ', '로', ' ', '받', '아', '서'], [' ', '쉽', '게', ' ', ' ', '토', '크', ' ', '나', ' ', '이', ' ', '저', ' ', '를', ' ', ' ', '사', '용', '할', ' ', ' ', '수', ' ', ' ', '있', ' ', ' ', '답', '니', '다']]
단일 문장 정수 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787, 6648]
여러 문장 정수 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544, 6513, 6590, 6536, 5852, 19, 5, 2639]]
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를 사용할 수 있습니다']
하위 단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를 사용할 수 있습니다']

- `SentencePieceProcessor` 클래스 통해 학습된 모델 불러올 수 있음
 - `tokenizer.load` 메서드를 통해 `petition_bpe.model` 모델 불러옴
- `encode_as_pieces` 메서드는 문장을 토큰화
- `encode_as_ids` 메서드는 토큰을 정수로 인코딩해 제공
 - 정수 데이터는 어휘 사전의 토큰에 매핑된 ID 값 의미
- 토크나이저 모델이나 자연어 처리 모델에서 나온 정수는 `decode_ids` 메서드나 `decode_pieces` 메서드를 통해 문자열 데이터로 변환 가능



#어휘 사전 불러오기

```
from sentencepiece import SentencePieceProcessor
```

```
tokenizer = SentencePieceProcessor()
```

```
tokenizer.load("petition_bpe.model")
```

```
vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
```

```
print(list(vocab.items())[:5])
```

```
print("vocab size :", len(vocab))
```

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '_이')]
```

```
vocab size : 8000
```

- `get_piece_size` 메서드는 센텐스피스 모델에서 생성된 하위 단어의 개수 반환
- `id_to_piece` 메서드는 정숫값을 하위 단어로 변환
 - 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구성
- 토큰 딕셔너리 출력의 `<unk>`, `<s>`, `</s>`
 - `<unk>` 토큰은 unknown의 약자로 OOV 발생 시 매핑되는 토큰
 - `<s>`, `</s>`는 문장의 시작 지점과 종료 지점을 표시하는 토큰
- 어휘 사전의 크기는 모델 학습 시 설정한 8,000으로 학습됨
 - 센텐스피스 라이브러리는 설정값에 따라 학습 방법이나 어휘 사전 크기가 달라짐

워드피스

- 워드피스(Wordpiece) 토크나이저
 - 바이트 페어 인코딩 토크나이저와 유사한 방법으로 학습되지만 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합
 - 학습 과정에서 확률적인 정보를 사용
 - 새로운 하위 단어 생성 시 이전 하위 단어와 함께 나타날 확률 계산해 가장 높은 확률로 하위 단어 선택

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x),f(y)}$$

- f는 빈도 나타내는 함수, x와 y는 병합하려는 하위 단어
- f(x, y)는 x와 y가 조합된 글자 쌍의 빈도 의미, 즉 sy 글자 쌍의 빈도
- score는 x와 y를 병합하는 것이 적절한지 판단하기 위한 점수가 됨



빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6),
('w', 'i', 'd', 'e', 's', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

'e'는 17번, 's'는 9번 등장 → score = 9/(17*9) = 0.06

'i'와 'd'가 각각 3번 등장 → score = 3/(3*3) = 0.33

→ 'i'와 'd' 쌍을 병합

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6),
('w', 'id', 'e', 's', 't', 3)

어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

(동일한 과정을 반복)

- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습

토큰나이저스

- 토큰나이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰나이저 구현, 학습 가능
- 허깅 페이스의 **토큰나이저스(Tokenizers)** 라이브러리 사용
- 토큰나이저스 라이브러리는 **정규화(Normalization)**와 **사전 토큰화(Pre-tokenixation)** 제공
 - 정규화
 - 일관된 형식으로 텍스트 표준화, 모호한 경우의 방지 위해 일부 문자 대체 혹은 제거
 - 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 두구점 처리, 특수 문자 처리 등
 - 사전 토큰화

- 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나눔
- 공백 혹은 구두점 기준으로 입력 문장을 나눠 텍스트 데이터 효율적으로 처리, 모델 성능 향상



```
#워드피스 토큰라이저 학습
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train("../datasets/corpus.txt")
tokenizer.save("../models/petition_wordpiece.json")
```

- 토큰라이저로 워드피스 모델을 불러와 정규화, 사전 토큰화 방식을 설정
- 정규화 방식
 - `normalizers` 모듈에 포함된 클래스 불러와 시퀀스(`Sequence`) 형식으로 인스턴스 전달
 - NFD 유니코드 정규화(`NFD`), 소문자 변환(`Lowercase`)을 사용함
- 사전 토큰화 방식
 - `pre_tokenizers` 모듈에 포함된 클래스 불러와 적용
 - 공백과 구두점을 기준으로 분리하는 방식 사용함
- 훈련(`train`) 메서드 통해 학습에 사용하려는 데이터셋 경로 전달, 학습 완료 됐다면 저장(`save`) 메서드로 학습 결과 저장

표 5.6 정규화 클래스

이름	설명
<code>NFD()</code>	NFD 유니코드 정규화
<code>NFKD()</code>	NFKD 유니코드 정규화
<code>NFC()</code>	NFC 유니코드 정규화
<code>NFKC()</code>	NFKC 유니코드 정규화
<code>Lowercase()</code>	입력 문장의 모든 대문자를 소문자로 변환
<code>Strip()</code>	입력 문장의 양 끝에 있는 공백 제거
<code>StripAccents()</code>	모든 엑센트 기호 제거
<code>Replace()</code>	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
<code>BertNormalizer()</code>	BERT 모델에 사용된 정규화 적용
<code>Sequence()</code>	여러 개의 정규화 모듈을 병합해 순서대로 수행

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_ ")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _(U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+!["^\\w\\s]+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior – removed: 삭제 – isolated: 개별 토큰 처리 – merged_with_previous: 이전 문자열과 결합해 토큰 처리 – merged_with_next: 다음 문자열과 결합해 토큰 처리 – contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리



```
#워드피스 토큰화
from tokenizers import Tokenizer
from tokenizers.decoders import WordPieceDecoder

tokenizer = Tokenizer.from_file("models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))

인코더 형식 : <class 'tokenizers.Encoding'>
단일 문장 토큰화 : ['안녕하세요', ',', '토', '##크', '##나이', '##저', '##가', '잘', '학습', '##되었', '##군요', '!']
여러 문장 토큰화 : [['이렇게', '입력', '##값을', '리스트', '##로', '받아서'], ['쉽게', '토', '##크', '##나이', '##저', '##를', '사용할', '수', '있다', '##니다']]
단일 문장 정수 인코딩 : [8760, 11, 8693, 8415, 16269, 7536, 7488, 7842, 15016, 8670, 8734, 0]
여러 문장 정수 인코딩 : [[8187, 19643, 13834, 28119, 7495, 12607], [9739, 8693, 8415, 16269, 7536, 7510, 14129, 7562, 8157, 7489]]
정수 인코딩에서 문장 변환 : 안녕하세요, 토큰라이저가 잘 학습되었군요!
```

- 모델 결과가 저장된 `petition_wordpiece.json` 파일 불러와 `Tokenizer` 객체 생성
 - `WordPieceDecoder()` 를 사용해 `Tokenizer` 의 디코더를 워드피스 디코더로 설정
 - 인코딩(`encode`) 메서드로 문장을 토큰화할 수 있으며, 인코딩 배치(`encode_batch`) 메서드로 여러 문장을 한 번에 토큰화할 수 있음
 - 둘 모두 워드피스 토큰라이저 통해 문장 토큰화하고 각 토큰의 색인 번호를 반환
 - 토큰화된 데이터는 토큰(`tokens`) 속성을 통해 값을 확인, 토큰 정수(`ids`) 속성으로 인코딩 된 문장의 ID 값을 출력
 - 정수를 다시 문장으로 변환하는 경우 디코딩(`decode`) 메서드를 통해 정수 인코딩된 결과를 다시 문장으로 디코딩해 출력
-
- 최근 연구 동향은 더 큰 말뭉치를 사용해 모델을 학습하고 OOV의 위험을 줄이기 위해 하위 단어 토큰화를 활용
 - 이외에도 유니코드 단위가 아닌 바이트 단위에서 토큰화하는 **바이트 수준 바이트 페어 인코딩(Byte-level Byte-Pair-Encoding, BBPE)**이나 크기가 큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습하는 **유니그램(Unigram)** 등이 있음

6. 임베딩

- 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 **텍스트 벡터화 (Text Vectorization)** 과정 필요
- **텍스트 벡터화(Text Vectorization)**
 - 텍스트를 숫자로 변환하는 과정
 - 기초적 텍스트 벡터화로 **원-핫 인코딩(One-Hot Encoding)**, **빈도 벡터화(Count Vectorization)** 등 있음
- 원 핫 인코딩
 - 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 0으로 표시하는 방식

Ex)

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

- 빈도 벡터화
 - 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

Ex) apples가 총 4번 등장 → 벡터값은 4

- 단어나 문장을 벡터 형태로 변환하기 쉽고 간단
- 단점
 - 벡터의 **희소성(Sparsity)**이 큼
 - 벡터 차원에 비해 토큰의 수가 적기 때문에 컴퓨팅 비용 증가나 차원의 저주 발생
 - 텍스트의 벡터가 입력 텍스트의 의미 내포하지 않으므로 의미적으로 유사한 문장이어도 벡터가 유사하게 나타나지 않을 수 있음

→ **워드 임베딩(Word Embedding)** 기법 사용해 해결

- **워드 임베딩(Word Embedding)**

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법
- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간 관계 추론
- 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어려움
→ 인공 신경망 활용해 **동적 임베딩(Dynamic Embedding)** 기법 사용

언어 모델

• 언어 모델(Language Model)

- 입력된 문장으로 각 문장을 생성할 수 있는 확률 계산하는 모델
- 주어진 문장을 바탕으로 문맥 이해하고 문장 구성에 대한 예측 수행
- 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용

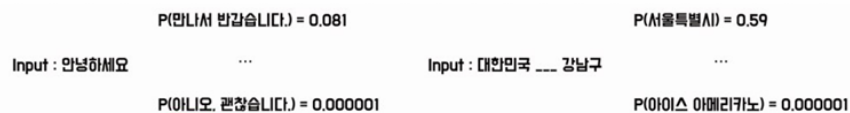


그림 6.1 언어 모델

자기회귀 언어 모델

• 자기회귀 언어 모델(Autogressive Language Model)

- 입력된 문장들의 조건부 확률 이용해 다음에 올 단어 예측
- 언어 모델에서 조건부 확률
 - 이전 단어들의 시퀀스 주어졌을 때, 다음 단어의 확률 계산하는 것
 - 이전에 등장한 모든 토큰의 정보 고려, 문장의 문맥 정보 파악해 다음 단어를 생성
 - 다음 단어는 다시 이전 단어를 기반으로 예측, 이 과정을 반복

수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 이전에 등장한 단어 시퀀스($w_1, w_2, \dots, w_{i-1}$)를 기반으로 다음 단어(w_i)의 확률 계산
- 조건부 확률의 연쇄법칙 적용

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

- 하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률 이용해 계산하는 방식
- 이전 단어들의 시퀀스가 주어졌을 때, 다음에 등장하는 단어의 확률을 이전 단어들의 조건부 확률 이용해 계산
- 문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱

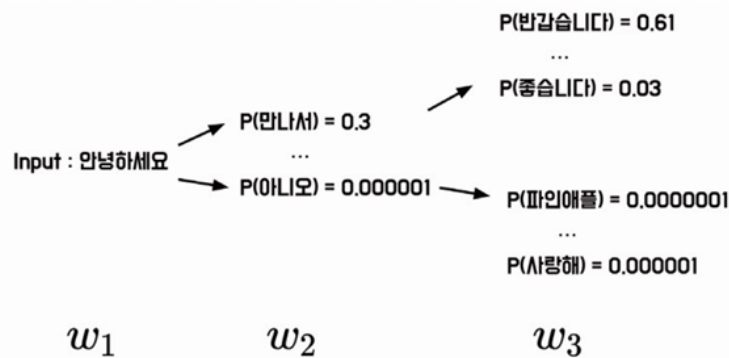


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
 - 입력 토큰 w_1 을 바탕으로 다음 토큰 w_2 가 등장할 확률 $P(w_2 | w_1)$ 을 예측
 - 그 다음 모델의 출력값 w_2 가 다시 입력값이 되어 확률 $P(w_3 | w_1, w_2)$ 예측
- 이렇게 모델의 출력값이 모델의 입력값으로 사용되는 특징으로 인해 자기회귀라 불림

통계적 언어 모델

- 통계적 언어 모델(Statistical Language Model)
 - 언어의 통계적 구조를 이용해 문장의 단어나 시퀀스를 생성하거나 분석
 - 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측

- 일반적으로 **마르코프 체인(Markov Chain)**을 이용해 구현
- **마르코프 체인(Markov Chain)**
 - 빈도 기반의 조건부 확률 모델
 - 이전 상태와 현재 상태 간의 전이 확률 이용해 다음 상태 예측
 - 주어진 데이터에서 각 변수가 발생한 빈도수 기반으로 확률을 계산

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 문맥 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성
- 한 빈도 등장한 적 없는 단어나 문장은 정확한 확률 예측 어려움
 - 관측한 적 없는 데이터를 예측하지 못하는 **데이터 희소성(Data sparsity)** 문제
- 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴 찾아 확률 분포 생성하므로 이를 이용해 새로운 문장 생성, 다양한 종류의 텍스트 데이터 학습 가능
- 통계적 언어 모델은 대규모 자연어 데이터 처리에 효과적
- 최근 연구되는 자연어 처리 기법은 언어 모델 활용해 가중치를 사전 학습
Ex)

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스크된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스크된

N-gram

- **N-gram**

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률 추정
- N-gram 모델은 입력 텍스트를 하나의 토큰 단위가 아닌 N개의 토큰을 묶어서 분석
 - 연속된 N개의 단어를 하나의 단위로 취급하여 추론
 - N이 1이면 **유니그램(Unigram)**, 2이면 **바이그램(Bigram)**, 3이면 **트라이그램(Trigram)**

Unigram : <u>안녕하세요</u> 만나서 진심으로 반가워요	안녕하세요. 만나서. 진심으로. 반가워요
Bigram : <u>안녕하세요 만나서</u> 진심으로 반가워요	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : <u>안녕하세요 만나서 진심으로</u> 반가워요	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

- N-1개의 토큰만 고려해 확률을 계산

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- w_i 는 예측하려는 단어, w_{i-1} 부터 w_{i-N+1} 까지는 예측에 사용되는 이전 단어
- 이전 단어들의 개수 결정하는 N의 값 조정하여 모델의 성능 조절



```
#N-gram
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

```
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

- 간단한 파이썬 코드 또는 NLTK 라이브러리로 구현 가능
- 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석하는 데 큰 효과, 관용적 표현 분석에도 활용
- 시퀀스에서 연속된 n개의 단어 추출하므로 단어의 순서 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

TF-IDF

- **TF-IDF(Term Frequency-Inverse Document Frequency)**
 - 텍스트 문서에서 특정 단어의 중요도 계산하는 방법

- **BoW(Bag-of-Words)**에 가중치를 부여하는 방법
- BoW
 - 문서나 문장을 단어의 집합으로 표현하는 방법
 - 등장하는 단어의 중복을 허용해 빈도를 기록

Ex)

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치 가짐

단어 빈도

- **단어 빈도(Term Frequency, TF)**
 - 문서 내에서 특정 단어의 빈도수 나타나는 값

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

- 해당 단어의 상대적인 중요도 측정하는 데 사용
- 단순히 단어의 등장 빈도수 계산하기 때문에 단어가 특정 문서에서 자주 사용되는 전문 용어나 관용어일 수도, 문서의 길이가 길어져 해당 단어의 TF 값이 높아질 수도 있음

문서 빈도

- **문서 빈도(Document Frequency, DF)**
 - 한 단어가 얼마나 많은 문서에 나타나는지를 의미
 - 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수 계산

수식 6.6 문서 빈도

$$DF(t, D) = \text{count}(t \in d: d \in D)$$

- DF는 단어가 몇 개의 문서에서 등장하는 지 계산
 - 높으면 특정 단어는 많은 문장에서 등장, 일반적으로 널리 사용되며 중요도가 낮다 생각할 수 있음
 - 낮으면 특정 단어가 적은 수의 문서에만 등장하므로 특정한 문맥에서만 사용된 단어이며 중요도 높을 수 있음

역문서 빈도

- 역문서 빈도(Inverse Document Frequency, IDF)
 - 전체 문서 수를 문서 빈도로 나눈 후 로그를 취한 값
 - 문서 내에서 특정 단어가 얼마나 중요한지 나타냄
 - 문서에서 특정 단어의 등장 횟수가 적으면 IDF는 상대적으로 커짐

수식 6.7 역문서 빈도

$$IDF(t, D) = \log\left(\frac{\text{count}(D)}{1 + DF(t, D)}\right)$$

TF-IDF

- 앞선 문서 빈도와 역문서 빈도를 곱한 값

수식 6.8 TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

- 문서 내에 단어가 자주 등장하지만 전체 문서 내에서 해당 단어가 적게 등장한다면 TF-IDF 값은 커짐
→ 전체 문서에서 자주 등장할 확률 높은 관사나 관용어 등의 가중치는 낮아짐
- 파이썬의 사이킷런(Scikit-learn) 라이브러리 활용해 TF-IDF 계산
 - 사이킷런 라이브러리는 말뭉치를 쉽게 TF-IDF 형태로 변환할 수 있게 TF-IDF 클래스 제공



#TF-IDF 클래스

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(  
    input="content",  
    encoding="utf-8",  
    lowercase=True,  
    stop_words=None,  
    ngram_range=(1, 1),  
    max_df=1.0,  
    min_df=1,  
    vocabulary=None,  
    smooth_idf=True,  
)
```

◦ TF-IDF(**TfidfVectorizer**) 클래스

◦ 입력값(**input**)

- 입력될 데이터의 형태 의미
- 기본값 content는 문자열 데이터 혹은 바이트 형태의 입력값

◦ 인코딩(**encoding**)

- 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값

◦ 소문자 변환(**lowercase**)

- 은입력받은 데이터를 소문자로 변환할지 여부
- 참값으로 설정하면 모든 입력 텍스트를 소문자로 변환

◦ 불용어(**stop_words**)

- 분석에 도움이 되지 않는 의미 없는 단어들을 의미, 입력받은 단어들은 단어 사전에 추가되지 않음

◦ N-gram 범위(**ngram_range**)

- 사용할 N-gram의 범위를 의미
- (최솟값, 최댓값) 형태로 입력, (1, 1)은 유니그램, (1, 2)는 유니그램과 바이그램을 사용한다는 것을 의미

◦ 최댓값 문서 빈도(**max_df**)

- 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리
- 정수 입력하면 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리하며, 1 이하의 실수를 입력하면 해당 비율을 초과해 등장한 단어

를 불용어 처리

- **최솟값 문서 빈도(`min_df`)**

- 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리한다는 뜻, 최댓값 문서 빈도와 동일한 패턴으로 입력

- **단어사전(`vocabulary`)**

- 미리 구축한 단어사전이 있다면 해당 단어 사전을 사용, 입력하지 않는다면 TF-IDF 학습 시 자동으로 구축

- **IDF 분모처리(`smooth_idf`)**

- 수식 6.7과 같이 IDF 계산 시 분모에 1을 더함



```
#TF-IDF 계산
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)

[[0.          0.          0.39687454 0.39687454 0.          0.79374908
  0.2344005 ]
 [0.61980538 0.          0.          0.          0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.          0.          0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```

- **TF-IDF** 클래스를 통해 **tfidf_vectorizer** 객체 불러오고, **fit** 메서드를 통해 준비된 말뭉치 학습
 - 객체 학습 완료되면 **transform** 메서드를 이용해 데이터 변환 수행, 또는 **fit_transform** 메서드를 통해 학습과 변환을 동시에 수행
 - **toarray** 메서드
 - TF-IDF를 넘파이 배열로 변환하며, (문서 수)×(단어 수)의 형태를 가짐
 - 각 행은 하나의 문서에 해당, 각 열은 단어를 의미
 - **vocabulary_** 속성은 TF-IDF에 사용된 단어 사전을 의미
 - 딕셔너리의 키는 고유한 단어, 값은 해당 단어에 대한 색인 값을 의미
-
- TF-IDF에서 점수가 가장 높은 값을 세 개만 추려 색인으로 정리하면
[[2, 3, 5], [0, 4, 6], [0, 1, 4]] 가 됨.
 - 이를 단어 사전과 매핑하면
[[famous, is, movie], [actor, like, that], [actor, don, like]] 가 됨

→ 이를 통해 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있음

- 빈도 기반 벡터화는 문장의 순서나 문맥 고려 X
 - 문장 생성과 같이 순서가 중요한 작업에는 부적합
- 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미 담고 있지는 않음